* Module 01 -- Hirely Project Story & Pitch Deck

CONFIDENTIAL & PRIVATE STUDY MATERIAL
Target Roles: AI Engineer, Applied AI Engineer, Backend AI Engineer
<- Back to Study Notes Index - Next Module: Project Explanation in Hinglish ->

[INDEX] Table of Contents

- 1. Executive Summary & Purpose
- 2. Problem Statement & Target Personas
- 3. Why Conventional Job Portals Fall Short
- 4. The Hirely Solution & Major Capabilities
- 5. End-to-End Recruitment Workflows
- 6. High-Level System Architecture
- 7. Tech Stack Justification & Engineering Choices
- 8. Critical Engineering Challenges & Trade-Offs
- 9. Architectural Limitations & Production Evolution
- 10. Interview Pitch: 60-Second Elevator Introduction
- 11. Interview Pitch: 2 - 3 Minute Technical Deep Dive
- 12. Interviewer Follow-Up Probes & Steering Strategies

1. Executive Summary & Purpose

Hirely is a dual-sided, startup-grade AI recruitment platform built to connect job-seeking **Candidates** with hiring **Recruiters** through intelligent discovery, deterministic candidate-job matching, and grounded AI assistance.

The primary objective of Hirely is to eliminate the friction, opacity, and screening fatigue of modern hiring. Instead of treating Generative AI as an autonomous decision-maker, Hirely uses a **hybrid architecture**:

1. Critical business calculations, security access boundaries, and numerical matching scores are kept strictly **deterministic** in backend Python code.
2. Large Language Models (Google Gemini) and vector embeddings are leveraged for **semantic understanding, contextual qualitative feedback, and evidence-grounded resume analysis**.

2. Problem Statement & Target Personas

* Persona 1: The Candidate (Job Seeker)

- **Application Blindness:** Submitting resumes to online job portals feels like shouting into a void. Candidates have no objective insight into whether their resume matches ATS filters.
- **Skill Gaps:** Applicants rarely receive feedback detailing the specific technical skills or experience bullets they are missing for a role.
- **Application Fatigue:** Manually rewriting bullet points and tailoring cover letters for dozens of openings is exhausting and leads to low-quality submissions.

* Persona 2: The Recruiter (Talent Acquisition Specialist)

- **Screening Overload:** A single job opening frequently attracts hundreds of resumes. Manual screening is slow, subjective, and prone to human cognitive fatigue.
- **Keyword Traps:** Traditional Applicant Tracking Systems rely on exact keyword matches. They fail when a candidate describes experience semantically without using the exact buzzword.
- **Unverified Claims:** Recruiters spend hours scanning multi-page documents trying to verify whether a candidate actually possesses hands-on experience with specific frameworks.

3. Why Conventional Job Portals Fall Short

Dimension	Conventional Job Portals (e.g., Indeed, Naukri)	Pure LLM Wrappers (Naive AI Demos)	Hirely Hybrid Platform
Matching Logic	Boolean keyword matching (misses synonyms and contextual experience).	Language model asked to "rate this resume 1-100" (random, non-reproducible, hallucinatory).	**Deterministic skill weighting (60%) combined with semantic vector cosine similarity (30%) and completeness (10%).**
Recruiter Review	Manual visual parsing of multi-page PDF documents.	Opaque AI summaries that cannot cite their sources.	**RAG Evidence Assistant with verbatim quote citations and substring verification.**
Candidate Help	None; pure job-listing bulletin board.	Generic ChatGPT wrapper prompts without job context.	**Job-bounded Cover Letter generation, ATS gap auditing, and structured bullet rewrites.**
Security & Privacy	Centralized, often sells user data.	Leaks candidate resumes and prompts to third-party logs.	**Strict privacy-preserving observability: zero prompt/PII logging, local/R2 storage.**

4. The Hirely Solution & Major Capabilities

Hirely structures the hiring lifecycle into two dedicated experiences supported by a centralized intelligence engine:

A. Candidate Experience

1. **Multi-Resume Management:** Candidates can upload multiple resumes (PDFs), parse them using Docling, inspect extracted structured JSON, and select an "active" resume for applications.
2. **Hybrid ATS Resume Analyzer:** Evaluates an active resume against any open job posting, generating a 0 - 100 score broken down by required skill coverage (45%), preferred skill coverage (15%), semantic embedding relevance (30%), and resume completeness (10%).
3. **AI Resume Improvement:** Diagnoses missing competencies and provides structured before-and-after bullet rewrites with explainable rationales.
4. **AI Cover Letter Generator:** Drafts highly targeted, professional cover letters grounded strictly in the candidate's verified resume experience and target job requirements.
5. **AI Career Coach:** Context-aware career advice helping candidates plan learning roadmaps and interview preparation.

B. Recruiter Experience

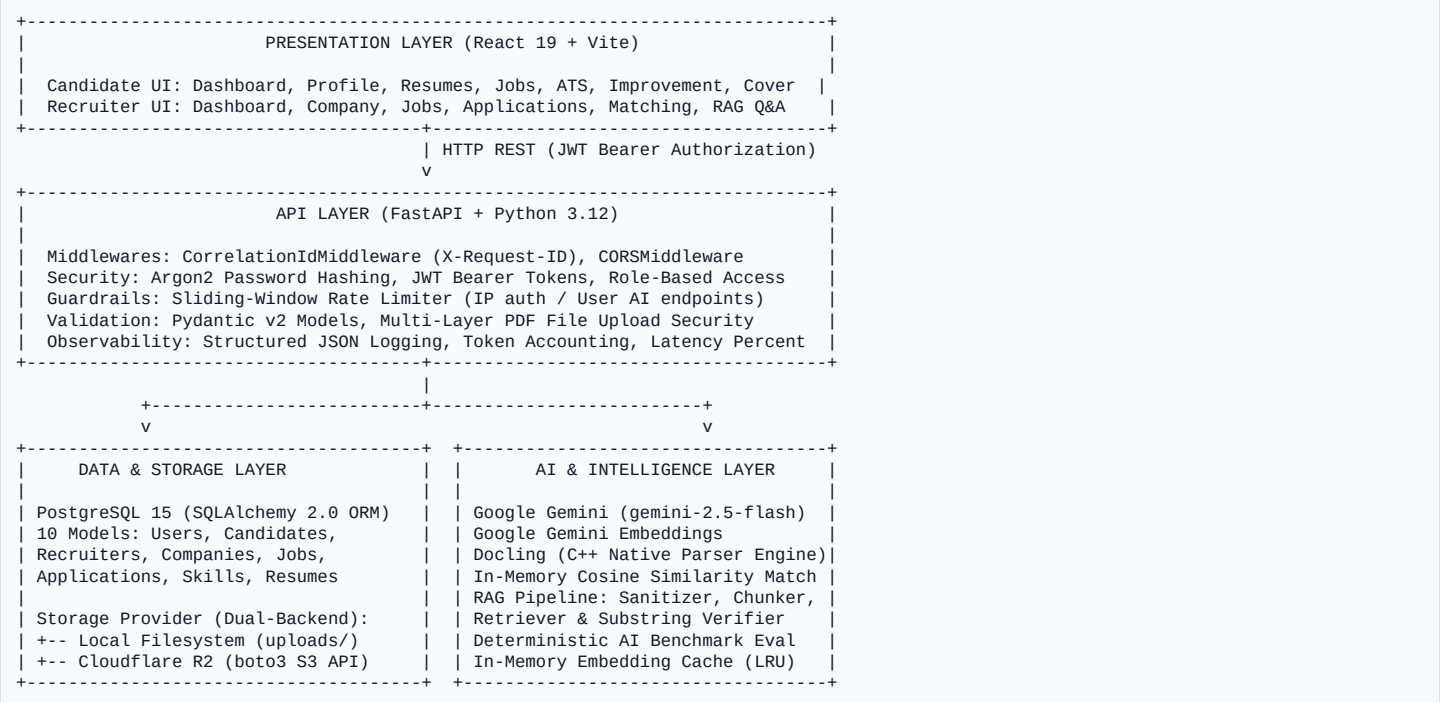
1. **Job Studio & Lifecycle:** Recruiters create, manage, and close job postings with explicit required and preferred skills.
2. **AI Job Description Analyzer:** Audits draft job postings before publication for clarity, structural completeness, and skill clarity.
3. **Two-Sided Matching & Ranking Engine:** Automatically ranks applicant pools for a job posting using deterministic skill matches and embedding similarity.
4. **AI Match Explanation:** Generates qualitative summaries explaining candidate strengths, potential red flags, and recommended interview questions.
5. **Candidate Evidence Assistant (RAG):** Interactive conversational assistant enabling recruiters to ask natural language questions about a candidate's resume, receiving answers grounded by verified verbatim citations.

5. End-to-End Recruitment Workflows

```
================================================================================
                              CANDIDATE LIFECYCLE
================================================================================
Register / Login (JWT)
       |
       v
Profile Setup (Headline, Bio, Location) -> Add Skills Tagging
       |
       v
Upload Resume (PDF) -> File Validation (Magic Bytes, ZIP-bomb protection, 10MB limit)
       |
       v
Docling Text Extraction -> Structured JSON Parsing -> Auto-Sync Skills -> Set as Active
       |
       v
Discover Active Jobs -> Run Hybrid ATS Analysis -> Review AI Resume Improvements
       |
       v
Generate Grounded Cover Letter -> Apply to Job (Unique constraint prevents duplicates)
       |
       v
Track Application Status (`applied` -> `shortlisted` -> `interview` -> `hired` / `rejected`)


================================================================================
                              RECRUITER LIFECYCLE
================================================================================
Register / Login (JWT) -> Create Recruiter Profile & Company Association
       |
       v
Draft Job Posting -> Run AI Job Description Analyzer (Fix ambiguities) -> Publish Job
       |
       v
Receive Candidate Applications -> Trigger Automated Candidate Matching & Ranking
       |
       v
Inspect Candidate Scores (Deterministic Breakdown + Vector Cosine Similarity)
       |
       v
Read AI Match Explanations (Strengths, Missing Skills, Suggested Interview Questions)
       |
       v
Launch Candidate Evidence Assistant (RAG):
Query Candidate Background -> Chunk Retrieval -> Grounded LLM Response with Verified Citations
       |
       v
Advance Application Status (`applied` -> `shortlisted` -> `interview` -> `hired` / `rejected`)
```

6. High-Level System Architecture



7. Tech Stack Justification & Engineering Choices

Component	Selected Technology	Why Chosen?	Key Trade-Off / Alternative Considered
Backend	**Python 3.12 + FastAPI**	Native async ASGI capabilities, first-class Pydantic v2 data validation, fast execution speed, and comprehensive AI/ML library support.	Django was considered too monolithic; Flask lacked native async and typed schema enforcement.
Database	**PostgreSQL 15 + SQLAlchemy 2.0**	Robust ACID transactional guarantees, strong relational foreign keys, schema migration support with Alembic, and future compatibility with `pgvector`.	MongoDB lacked strict multi-table relational integrity (e.g. cascading deletes between jobs, skills, and applications).
Migrations	**Alembic**	Version-controlled, reproducible database schema evolution. Runs automatically during containerized startup.	Manual SQL scripts or ORM `create_all()` are unversioned and hazardous in production.
Document Parser	**Docling (`langchain-docling`)**	Deep learning and C++ native document layout parsing. Excels at extracting multi-column resumes, tables, and structured text.	PyPDF2 / pdfminer often jumble multi-column text layouts, breaking chronological work history.
LLM Provider	**Google Gemini (`gemini-2.5-flash`)**	Large context window, fast completion speed, low token costs (\$0.075 / 1M prompt), structured JSON mode enforcement, and high reasoning fidelity.	OpenAI GPT-4o is significantly more expensive; local Ollama/Llama-3 models lacked consistent structured JSON adherence.
Embeddings	**Gemini Embeddings (`gemini-embedding-2`)**	768-dimensional dense representations optimized for semantic similarity and cross-document retrieval.	Sentence-Transformers required heavy local GPU/CPU RAM in Docker; Gemini embedding API is lightweight and fast.
Storage	**Dual Storage (Local + Cloudflare R2)**	Local disk is ideal for zero-cost single-node development; Cloudflare R2 provides zero-egress S3-compatible cloud object storage for stateless hosting.	AWS S3 has punitive egress fees; local-only storage fails on ephemeral container hosts (like Render or Fly.io).

8. Critical Engineering Challenges & Trade-Offs

1. Hybrid ATS Scoring vs. Pure LLM Scoring

- **The Challenge:** If you ask an LLM to "score this resume out of 100", it gives inconsistent, non-reproducible numbers (e.g., 85 on Monday, 62 on Tuesday).
- **The Solution:** Hirely uses **backend deterministic weighting**:

`Score = (0.45 x ReqSkills) + (0.15 x PrefSkills) + (0.30 x SemanticSimilarity) + (0.10 x Completeness)`

The LLM is restricted to qualitative feedback (explaining strengths and weaknesses), while the score itself is 100% reproducible.

2. Guarding RAG Against Hallucinated Experience

- **The Challenge:** In recruitment, an LLM might claim a candidate has "7 years of AWS experience" because the job description asked for it, even if the resume only mentions a 2-month tutorial.

- **The Solution:** Hirely's RAG Evidence Assistant enforces **strict quote verification**. The prompt instructs the LLM to return exact quotes supporting its answer. Before the API response is returned, the backend verifies that the returned quote exists as a verbatim substring in the retrieved resume chunks.

3. Asymmetric Process-Isolated Parsing Limits

- **The Challenge:** Docling utilizes a native C/C++ engine (`docling-parse`). In CPython, native C calls cannot be interrupted via Python thread timeouts or signals. A maliciously malformed PDF could lock a CPU core.
- **The Solution:** Implemented strict pre-parsing file validation: magic byte verification (`%PDF-`), ZIP structure inspection, 10 MB upload cap, 50 MB uncompressed limit, and maximum 10-page bounds (`max_num_pages=10`).

4. Database Connection Pool Exhaustion Across Workers

- **The Challenge:** When deploying multiple Uvicorn worker processes in Docker, each worker instantiates its own SQLAlchemy connection pool. Uncoordinated pool sizing causes PostgreSQL `FATAL: remaining connection slots are reserved for non-superuser connections`.
- **The Solution:** Formulated strict connection budgeting:

```
Total Connections = (DB_POOL_SIZE + DB_MAX_OVERFLOW) x Workers
```

Configured `pool_size=5` and `max_overflow=10` across 4 workers = 60 connections, safely under PostgreSQL's default limit of 100.

---

9. Architectural Limitations & Production Evolution

When discussing Hirely in interviews, demonstrate senior engineering maturity by being completely honest about current architectural boundaries:

1. **Process-Local Rate Limiting & Metrics:** The sliding-window rate limiters and in-memory metrics collectors reside in Python process memory. In a clustered multi-node deployment, rate limit windows are not shared across workers.
   - *Production Evolution:* Introduce a shared **Redis** instance for distributed sliding-window rate limiting and export metrics to Prometheus.
2. **Synchronous Parser Execution:** Resume parsing runs synchronously within the request-response cycle.
   - *Production Evolution:* Offload parsing to an asynchronous task queue (**Celery** or **Redis Queue**) with dedicated worker subprocesses and OS-level `SIGKILL` timeouts.
3. **In-Memory Candidate Vector Matching:** Vector cosine similarity is calculated in-memory across applicants for a specific job.
   - *Production Evolution:* Enable **pgvector** inside PostgreSQL to index dense embeddings directly in the database with HNSW / IVFFlat indexes for cross-tenant search.

---

10. Interview Pitch: 60-Second Elevator Introduction

"Hirely is an AI-powered recruitment platform designed to solve the two biggest problems in modern hiring: candidates applying blindly without knowing their ATS fit, and recruiters drowning in hundreds of unverified resumes.
What makes Hirely unique is that we deliberately avoid naive LLM wrappers. We built a hybrid architecture using Python, FastAPI, and PostgreSQL. Critical business logic, role-based access control, and numerical candidate-job matching scores are completely deterministic in backend code. We leverage Google Gemini and dense vector embeddings specifically for what they excel at: semantic relevance, resume improvement, and an interactive RAG Evidence Assistant that verifies candidate claims with exact quote citations.
The entire system is containerized with Docker Compose, features automated schema migrations, and includes built-in privacy-preserving observability to prevent PII and prompt leakage."

---

11. Interview Pitch: 2 - 3 Minute Technical Deep Dive

"I'd like to walk you through the engineering design of Hirely. The project connects job seekers and recruiters through two distinct experiences backed by an intelligence layer.
On the candidate side, an applicant can manage multiple resumes. When a resume is uploaded, it passes through multi-layer file security -- verifying PDF magic bytes, preventing zip bombs, and enforcing file and page limits. We use Docling to extract structured text and automatically synchronize discovered skills into the candidate's profile.
When evaluating a resume against an opening, we built a hybrid ATS scoring engine. Rather than delegating scoring to a non-deterministic LLM, we use a backend weighted formula: 45% for required skills, 15% for preferred skills, 30% for semantic cosine similarity using Gemini embeddings, and 10% for resume completeness. The LLM is then used exclusively to produce grounded qualitative feedback: before-and-after bullet rewrites and tailored cover letters.
On the recruiter side, talent acquisition teams can audit their job descriptions before posting, automatically rank candidate pools, and use our RAG Candidate Evidence Assistant. Instead of skimming through a 5-page resume, a recruiter can ask questions like: 'Does this candidate have production Kubernetes experience?' Our RAG pipeline chunks the resume, retrieves the relevant context via vector similarity, and prompts Gemini to answer strictly based on the text, returning exact verbatim quote citations. Our backend verifies that those quotes exist in the source document to eliminate hallucination.
On the infrastructure side, we run on FastAPI with SQLAlchemy 2.0 and PostgreSQL 15, managed through Alembic migrations. We implemented dual storage support: local disk for zero-cost development, and Cloudflare R2 object storage via boto3 for stateless cloud hosting. We also built privacy-preserving observability: every request receives an X-Request-ID, logs are formatted as single-line JSON with PII and prompt scrubbing, and token usage and latency percentiles are tracked.
Our current architecture operates in-memory for rate limiting and embedding similarity, and in our next iteration, our roadmap focuses on introducing Redis for distributed rate limiting and Celery for asynchronous parser isolation."

---

12. Interviewer Follow-Up Probes & Steering Strategies

When the Interviewer Asks...	What They Are Really Testing	How You Should Answer (Steering Strategy)
"Why didn't you just let GPT-4 give the ATS match score?"	Do you understand LLM non-determinism, drift, and reproducibility?	Explain that LLM scoring is non-deterministic and arbitrary. A score of 72 might become 84 with temperature jitter. By keeping numerical weights in backend math and using embeddings for similarity, scores are 100% reproducible.
"How do you prevent the RAG assistant from hallucinating credentials?"	Retrieval grounding and verification engineering.	Walk through the quote verification pipeline: bounded context prompts, low temperature, and the backend substring check that programmatically asserts the returned quote exists in the retrieved chunks.
"How do you handle privacy when sending resumes to Google Gemini?"	Data compliance, PII handling, and API boundaries.	Explain that raw user contact info is parsed and separated; prompts contain only relevant experience text; structured logging regex-scrubs emails, phones, and tokens; and zero prompts or completions are logged to disk.
"What happens if your primary Gemini model has an outage?"	Resilience, retry policies, and graceful degradation.	Detail the fallback mechanism in `gemini_service.py`: transient retries with exponential backoff, followed by automatic failover from `gemini-2.5-flash` to `gemini-2.0-flash`.
"How does your Docker Compose setup handle database readiness?"	Container orchestration and race conditions.	Explain that the backend container doesn't just wait for the DB port; it depends on `hirely-postgres` passing its `pg_isready` healthcheck AND `hirely-migration` running `alembic upgrade head` and exiting with code 0.